数据结构与算法

东北大学软件学院
张莉

2025年9月

张莉

Office：信息B 447

E-mail:zhangl@swc.neu.edu.cn

关注微助教公众号

课程编号：PO870



为什么学习数据结构?

- 程序是用来处理数据，执行复杂计算的。
- 数据形式越来越复杂，数据量越来越大
- **合适的数据结构和算法设计**可以大幅度提高程序的性能
- 本课程主要讨论
  - 如何组织数据，建立合适的数据结构
  - 如何设计算法，提高程序执行的效率
  - 如何评估算法性能

课程简介

- 数据结构+算法=程序
- 数据结构：问题的数据模型
  - 线性结构：线性表、栈、队列、串、…..
  - 非线性结构：树、图
- 算法：求解问题的策略
  - 查找:
  - 排序

课程简介

课程性质： 专业技术基础课

先修课程： 离散数学、C/C++语言程序设计

学时安排： 总学时64学时（含16学时实验）

- 后续课：算法基础
  - 部分数据结构的内容移至算法基础
- 操作系统、数据库、编译原理….
- “**数据结构**涉及到各方面的知识，
  - 如计算机硬件范围的存储装置和存取方法；
  - 在计算机软件范围中的文件系统，数据的动态存储与管理，信息检索；数学范围的许多算法知识，
  - 还有一些综合性的知识，如编码理论、算子关系、数据类型、数据表示、数据运算、数据存取等各方面的知识。
- 因此，**数据结构**是数学、计算机硬件、软件三者之间的一门核心课程。

课程简介

成绩结构

- 平时成绩（作业，课堂测试，期中考试等）： 20%
- 实验（实验代码，报告）： 20%
- 期末考试:
  - 闭卷 60%

参考教材

- 《数据结构（C语言版）》

  严蔚敏 吴伟民编著 清华大学出版社





2 章节目录

第1章 绪论

第2章 线性表

第3章 栈和队列

第4章 串

第5章 数组和矩阵

第6章 树和二叉树

第7章 图

第9章 查找

第10章 排序

第一章 绪论

1.1 数据结构讨论的范畴

1.2 基本概念

1.3 算法和算法的量度



1.1 数据结构讨论的范畴

Niklaus Wirth:

Algorithm + Data Structures = Programs

程序设计: 为计算机处理问题编制一组指令集

算法: 处理问题的策略

数据结构: 问题的数学模型

例如: 数值计算的程序设计问题

结构静力分析计算

—— 线性代数方程组

全球天气预报

—— 环流模式方程
(球面坐标系)

数值计算的程序设计问题

- 数值计算解决问题的一般步骤：
- 数学模型→设计算法→编出程序→测试→最终解答
- 数值计算的关键是：如何
）？
- 程序设计人员比较关注

今有鸡兔同笼，上有三十五头，下有九十四足，问鸡兔各几何?

“鸡兔同笼”问题

- 二元一次方程组

全球天气预报

- 环流模式方程
(球面坐标系)

非数值计算的程序设计问题

数据元素之间的相互关系一般无法用数学方程加以描述

例一：学生信息检索系统

算法：？

模型：？

需要检索的项目

如何检索。 用户界面

各种表格

学号	姓名	性别	专　　业	年 级
980001	吴承志	男	计算机科学与技术	98 级
980002	李淑芳	女	信息与计算科学	98 级
990301	刘　丽	女	数学与应用数学	99 级
990302	张会友	男	信息与计算科学	99 级
990303	石宝国	男	计算机科学与技术	99 级
000801	何文颖	女	计算机科学与技术	2000 级
000802	赵胜利	男	数学与应用数学	2000 级
000803	崔文靖	男	信息与计算科学	2000 级
010601	刘　丽	女	计算机科学与技术	2001 级
010602	魏永鸣	男	数学与应用数学	2001 级

（a）学生信息表

崔文靖	8
何文颖	6
李淑芳	2
刘　丽	3，9
石宝国	5
魏永鸣	10
吴承志	1
赵胜利	7
张会有	4

（b）姓名索引表

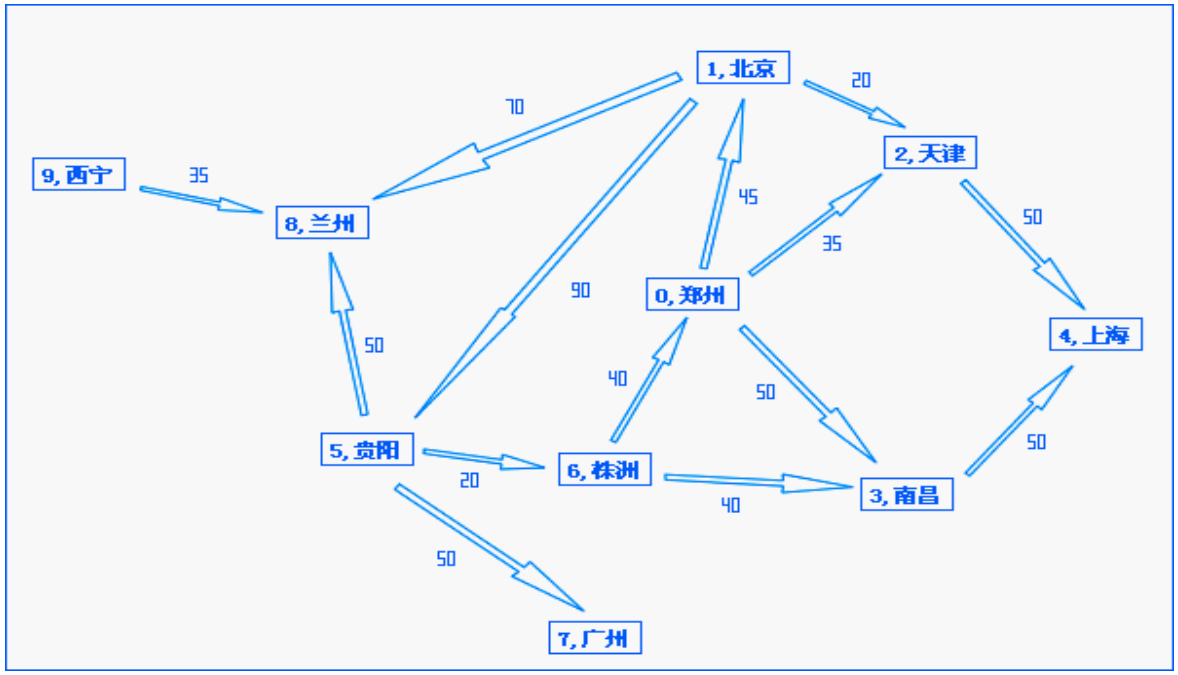
计算机科学与技术	1，5，6，9
信息与计算科学	2，4，8
数学与应用数学	3，7，10

（c）专业索引表

2000 级	6，7，8
2001 级	9，10
98 级	1，2，3
99 级	4，5

（d）年级索引表

例二：网络连通性问题



算法：？ 查找两点间是否有路，最短路等

模型：？ 地图

例三 书的摆放

- 如何把一些书放到书架上?
  - 多少本? 和问题大小相关
  - 10? 100? 1000? 10000?.....
  - 什么样的书架?
- 算法： 插入新的书，查找某本书





例三 书的摆放

- 如果架子上有很多书，我们通常需要做两个动作：插入新书，检索某个特定的书.
- 如何安排书使两种操作速度快?
  - 方法1：随机安排
  - 插入----将书放在尾部.
    - 查找----!#%$^



例三 书的摆放

- 如果架子上有很多书，我们通常需要做两个动作：插入新书，检索某个特定的书.
- 如何安排书使两种操作速度快?
  - 方法1：随机安排
  - 插入----将书放在尾部.
    - 查找----!#%$^



例三 书的摆放

- 方法 2: 按 字母序
- 插入---移动后面的书.
- 查找
  - Sequential
  - Binary search



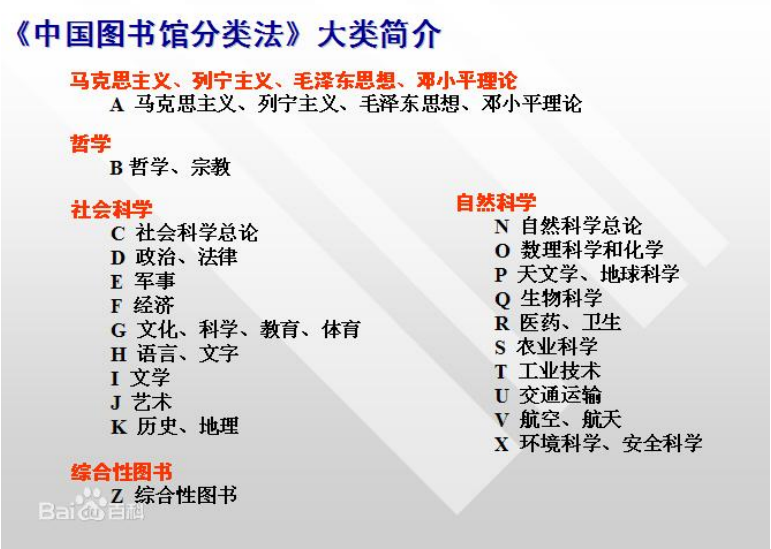
例三 书的摆放

- 方法 2: 按 字母序
- 插入---移动后面的书.
- 查找
  - Sequential
  - Binary search



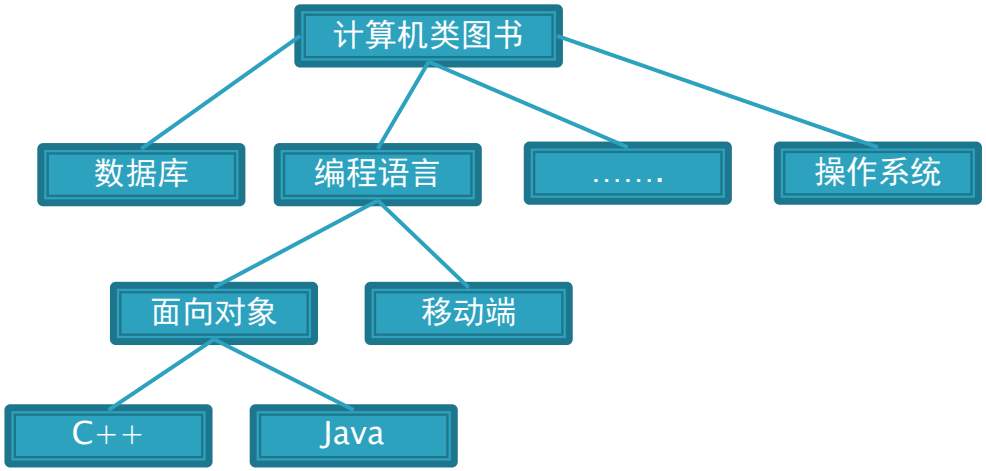
例三 书的摆放

- 方法 3: 按类别
- 插入---移动类别里的书
- 查找
  - 在类别内二分查找



例三 书的摆放

- 方法 3: 按类别



每类多少个架子?

- **求解非数值计算的问题：主要考虑的是设计出合适的数据结构及相应的算法。即：首先要考虑对相关的各种信息如何表示、组织和存储?**
- 对于一个给定的问题有很多种解决方案
- 每一种解决方案适用的情况不同，各有优缺点
- **数据结构是一门讨论“描述现实世界实体的数学模型(非数值计算)及其上的操作在计算机中如何表示和实现”的学科。**

1.2 基本概念

数据：有能**被输入**到计算机中，且能被计算机**处理的符号**的集合。**计算机操作的对象**的总称。是计算机处理的**信息的**某种特定的符号**表示形式**。它是信息的载体，它能够被计算机识别、存储和加工处理。

计算机科学中，所谓数据就是计算机加工处理的对象，它可以是数值数据，也可以是非数值数据。数值数据是一些整数、实数或复数，主要用于工程计算、科学计算和商务处理等；非数值数据包括字符、文字、图形、图像、语音等。

数据元素

是数据（集合）中的一个“个体”
是数据结构中讨论的**基本单位**。例如，学生信息检索系统中学生信息表中的一个记录、对弈问题中状态树的一个状态、排课问题中的一个顶点等，都被称为一个数据元素。

课程编号	课 程 名	学时
024002	程序设计基础	64
024010	汇编语言	48
024016	计算机原理	64
024020	数据结构	64
024021	微机技术	64
024024	操作系统	48
024026	数据库原理	48

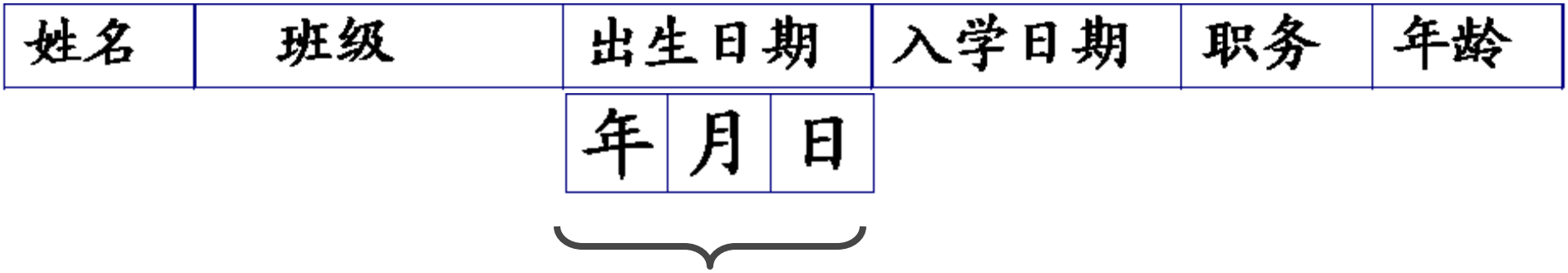
数据元素

数据项：

是数据结构中讨论的**最小**单位

数据元素可以是数据项的集合

例如：描述一个学生的数据元素可以是



称之为组合项

数据对象:

是性质相同的数据元素的集合，是数据的一个子集。 数据元素是数据对象的一个实例。

例如，整数数据对象是集合N={…-2,-1,0,1,2….}

字母字符数据对象是集合N={‘A’,’B’…,’Z’}

数据对象

课程编号	课 程 名	学时
024002	程序设计基础	64
024010	汇编语言	48
024016	计算机原理	64
024020	数据结构	64
024021	微机技术	64
024024	操作系统	48
024026	数据库原理	48

数据元素

数据项

数据结构：带**结构**的数据元素的集合

假设用**三个 4 位的十进制数**表示一个含 **12 位数的十进制数。**

例如:

3214,6587,9345 — **a1**(3214),**a2**(6587),**a3**(9345)

则在数据元素a1、a2 和 a3之间存在着**“次序”关系** **<a1,a2>、<a2,a3>**

3214，6587，9345 ≠ 6587，3214，9345

a1 a2 a3 a2 a1 a3

数据结构： 带**结构**的数据元素的集合

又例，在2行3列的二维数组{a1, a2, a3, a4, a5, a6}中六个元素之间存在两个关系:

a1	a2	a3
a4	a5	a6

行的次序关系:

$$row = \{<a1,a2>,<a2,a3>,<a4,a5>,<a5,a6>\}$$

列的次序关系:

$$col = \{<a1,a4>,<a2,a5>,<a3,a6>\}$$

a1 a3 a5
a2 a4 a6

$\neq$

a1 a2 a3
a4 a5 a6

数据结构： 带**结构**的数据元素的集合

再例，在一维数组 {a1, a2, a3, a4, a5, a6} 的数据元素之间存在如下的**次序关系**:

$$\{<a_i, a_{i+1}>| \ i=1, 2, 3, 4, 5\}$$

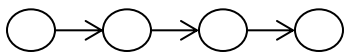
可见，不同的“**关系**”构成不同的“**结构**”

或者说，**数据结构**是**相互之间存在着某种逻辑关系的数据元素的集合。**

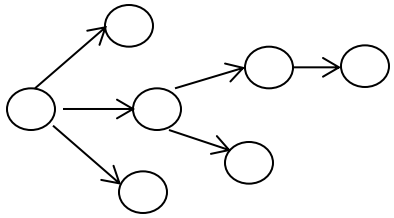
数据的逻辑结构

逻辑结构　　四类:

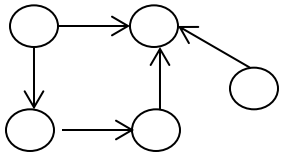
线性结构



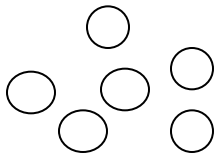
树形结构



图状结构



集合结构



数据结构的形式定义为:

数据结构是一个二元组

$$Data_Structures = (D, S)$$

其中:D 是**数据元素的有限集**,

S 是 D上**关系的有限集**。

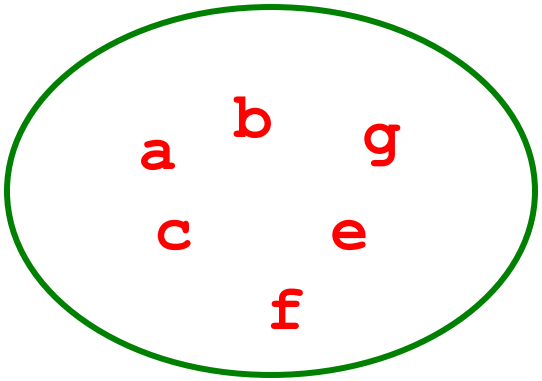
例如： S={D,R},D={a,b,c,d,e,f,g}

R1={φ}

R2={<a,b>,<b,c>,<c,d>,<d,e>,<e,f>,<f,g>}

R3={<a,b>,<a,c>,<a,d>,<b,e>,<b,f>,<c,g>}

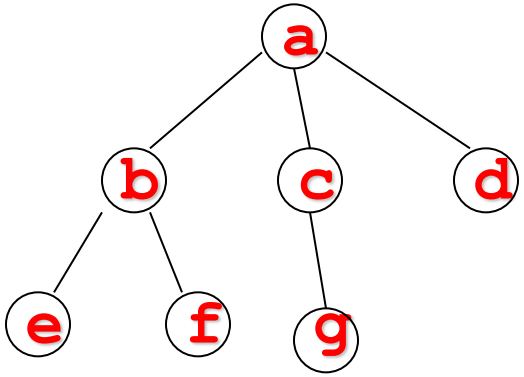
R4={<a,e>,<b,c>,<c,a>,<e,f>,<f,g>,<c,f>}



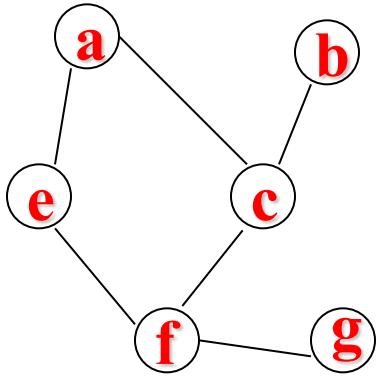
S1-集合



S2 - 线性结构



S3 - 树型结构



S4 - 图形结构

- 数据结构包括**逻辑结构**和**物理结构**。
- 数据的**逻辑结构**可以看作是从具体问题抽象出来的数学模型，它与数据的存储无关。
- 数据结构在计算机中的标识(又称映像)称为数据的**物理结构**，或称**存储结构**。它所研究的是数据结构在计算机中的实现方法，包括数据结构中元素的表示及元素间关系的表示。

数据的存储结构

—— 逻辑结构在存储器中的**映象**

存储结构两方面的内容

（1）“数据元素”的映象 ？

（2）“关系”的映象 ？

基本的存储方法

- 四种基本的存储方法：
  - （1）顺序存储方法
  - （2）链接存储方法
  - （3）索引存储方法
  - （4）散列存储方法

同一种逻辑结构可采用不同的存储方法（以上四种之一或组合），这主要考虑的是运算方便及算法的时空要求。

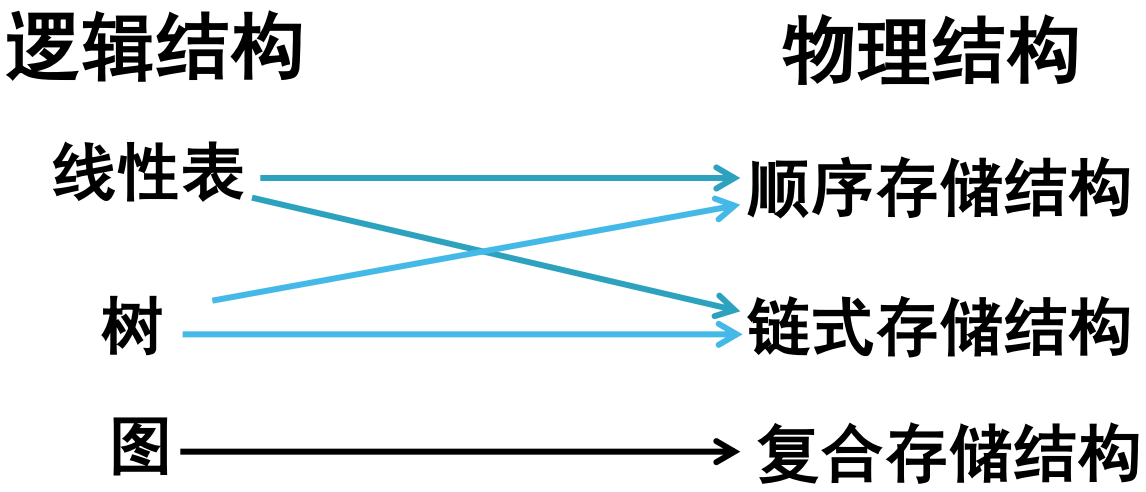


图1-4　逻辑结构与所采用的存储结构

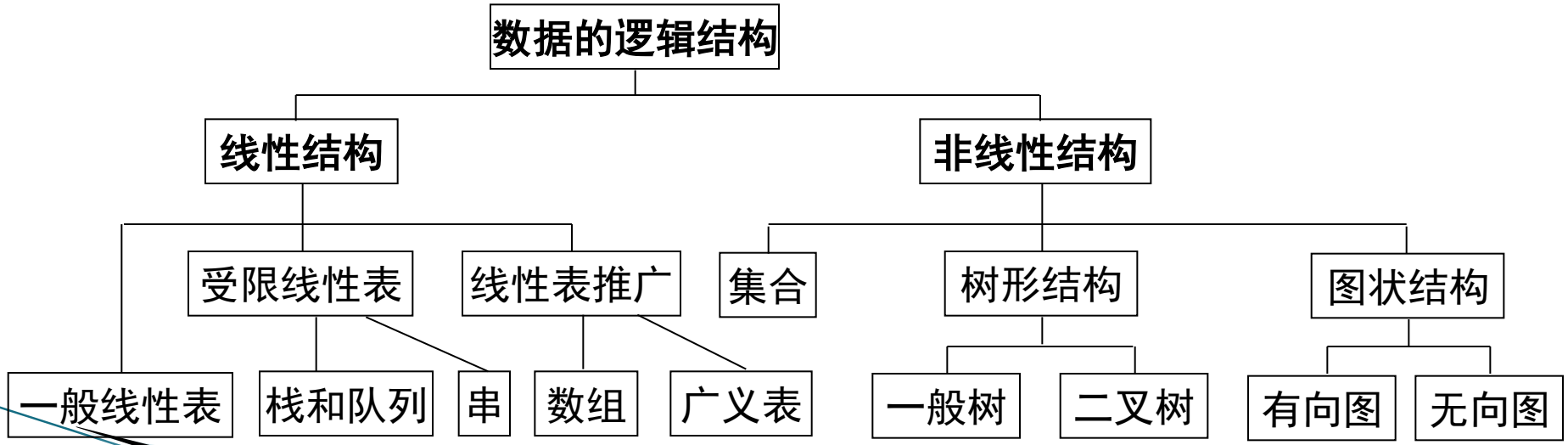


图1-5　数据逻辑结构层次关系图

数据元素的映象方法：

用二进制位(bit)的位串表示数据元素

$$(321)_{10} = (501)_8 = (101000001)_2$$

$$A = (101)_8 = (001000001)_2$$

关系的映象方法：

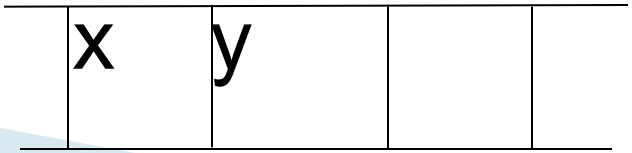
(表示<x, y>的方法)

顺序映象

以相对的存储位置表示后继关系

例如:令 y 的存储位置和 x 的存储位置之间差一个常量 C

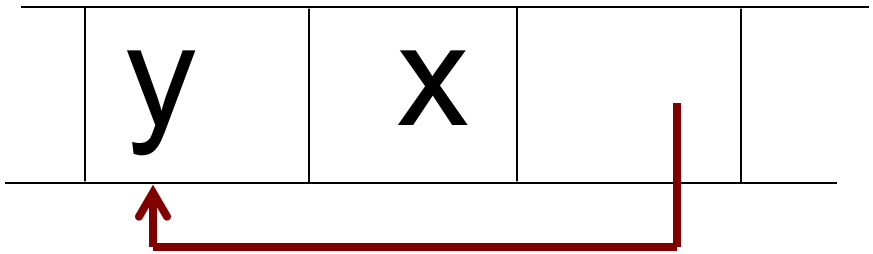
而 C 是一个隐含值，**整个存储结构中只含数据元素本身的信息**



链式映象

以附加信息(指针)表示后继关系

需要用一个和 x 在一起的**附加信息**指示 y 的存储位置



在不同的编程环境中，

存储结构可有不同的描述方法。

当用高级程序设计语言进行编程时，通常可用高级编程语言中提供的数据类型描述之。

数据类型

数据类型是用以刻划程序中操作对象的特性。在用高级语言编写的程序中，每个变量、常量或表达式都有一个它所属的确定的数据类型。

例如，C 语言中提供的**基本数据类型**

整型　int

浮点型　float

双精度型　double

} **实型（C++语言）**

字符型　char

逻辑型　bool　（C++语言）

类型显式地或隐含地规定了在程序执行期间变量或表达式所有可能的取值范围，以及在这些值上允许进行的操作。

数据类型 是一个 **值的集合**和定义在此集合上的**一组操作**的总称。

- 在高级程序设计语言中，数据类型可分为两类：
- **原子类型**
  - 值是不可分解的。
  - 如C语言中整型、字符型、浮点型、双精度型等基本类型，分别用保留字int、char、float、double标识。
- **结构类型**
  - 值是由若干成分按某种结构组成的，因此是可分解的，并且它的成分可以是非结构的，也可以是结构的。
  - 例如，数组的值由若干分量组成，每个分量可以是整数，也可以是数组等。

- 在某种意义上，数据结构可以看成是“一组具有相同结构的值”，而数据类型则可被看成是由一种数据结构和定义在其上的一组操作所组成的。

抽象数据类型

抽象数据类型(Abstract Data Type 简称ADT):

- 由用户定义，用以表示应用问题的数据模型
- 由基本的数据类型组成, 并包括一组相关的服务（或称操作）
- 数据封装和信息隐蔽，使用与实现相分离

- 抽象数据类型的定义取决于它的一组**逻辑特性**，而与其在计算机内部如何表示和实现无关。即不论其内部结构如何变化，**只要它的数学特性不变，都不影响其外部的使用。**

例如，ADT复数的定义：

ADT Complex {

数据对象：

D = {e1,e2 | e1,e2∈RealSet }

数据关系：

R1 = {<e1,e2> | e1是复数的实数部分

| e2 是复数的虚数部分 }

基本操作：

AssignComplex(&Z, v1, v2)

操作结果：构造复数 Z,其实部和虚部分别被赋以参数 v1 和 v2 的值。

DestroyComplex(&Z)

操作结果：复数Z被销毁。

GetReal(Z, &realPart)

初始条件：复数已存在。

操作结果：用realPart返回复数Z的实部值。

GetImag(Z, &ImagPart)

初始条件：复数已存在。

操作结果：用ImagPart返回复数Z的虚部值。

Add(z1,z2, &sum)

初始条件：z1, z2是复数。

操作结果：用sum返回两个复数z1, z2 的和值。

} ADT Complex

假设:z1和z2是上述定义的复数

则 Add(z1, z2, z3) 操作的结果

即为用户企求的结果

$$z3 = z1 + z2$$

ADT 有两个重要特征:

数据抽象：用ADT描述程序处理的实体时，强调的是其**本质的特征、其所能完成的功能**以及它和**外部用户的接口**（即**外界使用它的方法**）。

数据封装：将实体的**外部特性和其内部实现细节分离**，并且**对外部用户隐藏其内部实现细节**。

ADT 与数据类型的关系

- 抽象数据类型和数据类型实质上是一个概念。“抽象”的意义在于数据类型的数学抽象特性。
- 另一方面，抽象数据类型的范畴更广，它不再局限于各处理器中已定义并实现的数据类型，还包括用户在设计软件系统时自己定义的数据类型。

抽象数据类型的描述方法

抽象数据类型可用（D，S，P）三元组表示。

其中：D 是数据对象；

S 是 D 上的关系集；

P 是对 D 的基本操作集。

ADT 抽象数据类型名 {

数据对象： 〈数据对象的定义〉

数据关系： 〈数据关系的定义〉

基本操作： 〈基本操作的定义〉

} ADT 抽象数据类型名

其中基本操作的定义格式为:

基本操作名（参数表）

初始条件： 〈初始条件描述〉

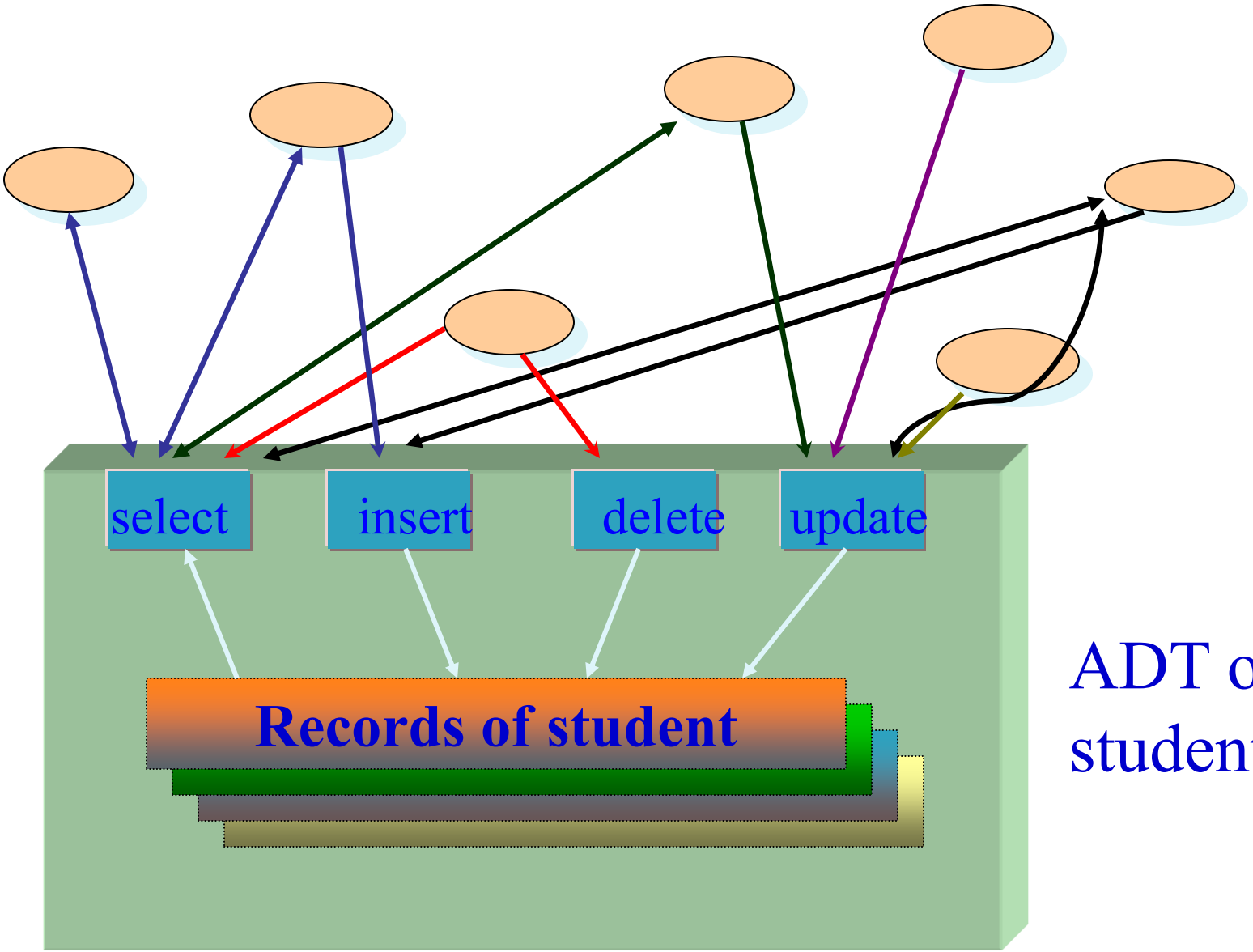
操作结果： 〈操作结果描述〉

赋值参数 只为操作提供输入值。

引用参数 以&打头，除可提供输入值外，还将返回操作结果。

初始条件 描述了操作执行之前数据结构和参数应满足的条件，若不满足，则操作失败，并返回相应出错信息。

操作结果 说明了操作正常完成之后，数据结构的变化状况和应返回的结果。



ADT of students

抽象数据类型的表示和实现

抽象数据类型需要通过**固有数据类型**(高级编程语言中已实现的数据类型)来实现。

例如，对以上定义的复数。

// -----存储结构的定义

```
typedef  struct {
    float realpart;
    float imagpart;
}complex;
```

// -----基本操作的函数原型说明

```
void Assign( complex &Z,
                    float realval, float imagval );
```

// 构造复数 Z,其实部和虚部分别被赋以参数 // realval 和 imagval 的值

```
float GetReal( cpmplex Z );
  // 返回复数 Z 的实部值

float Getimag( cpmplex Z );
  // 返回复数 Z 的虚部值

void add( complex z1, complex z2,
                              complex  &sum );
 // 以 sum 返回两个复数 z1, z2 的和
```

// -----基本操作的实现

```
void add( complex z1, complex z2,
                                complex  &sum ) {
 // 以 sum 返回两个复数 z1, z2 的和
  sum.realpart = z1.realpart + z2.realpart;
    sum.imagpart = z1.imagpart + z2.imagpart;
}
```

{ 其它省略 }



1.3 算法和算法的衡量

一、算法特性

二、算法设计的原则

三、算法效率的衡量方法和准则

四、算法的存储空间需求

一、算法特性

算法是对特定问题求解步骤的一种描述，是指令的有限序列。其中每一条指令表示一个或多个操作。。一个算法必须满足以下**五个重要特性**：

1．**有穷性** 对于任意一组合法输入值，在执行**有穷步骤**之后一定能结束，即：
算法中的每个步骤都能在**有限时间**内完成。

2．**确定性** 对于**每种情况**下所应执行的操作，在算法中都有**确切**的规定，使算法的执行者或阅读者都能明确其含义及如何执行。**并且在任何条件下，算法都只有一条执行路径。对于相同的输入只能得出相同的输出**

3．可行性 算法中的所有操作都必须足够基本，都可以通过已经实现的基本操作运算有限次实现之。

4．有输入 作为算法加工对象的量值，通常体现为算法中的一组变量。有些输入量需要在算法执行过程中输入，而有的算法表面上可以没有输入，实际上已被嵌入算法之中。

5．有输出 它是一组与“输入”有确定关系的量值，是算法进行信息加工后得到的结果，这种确定关系即为算法的功能。

算法的描述

算法可以使用各种不同的方法来描述。

- **自然语言**。优点是简单且便于人们对算法的阅读。缺点是不够严谨。
- **程序流程图**、N-S图等算法描述工具。其特点是描述过程简洁、明了。

用以上两种方法描述的算法不能够直接在计算机上执行，若要将它转换成可执行的程序还有一个编程的问题。

- **某种程序设计语言**来描述算法，不过直接使用程序设计语言并不容易，而且不太直观，常常需要借助于注释才能使人看明白。
- 为了解决理解与执行这两者之间的矛盾，使用**伪码语言**。伪码语言介于高级程序设计语言和自然语言之间，它忽略高级程序设计语言中一些严格的语法规则与描述细节，因此它比程序设计语言更容易描述和被人理解，而比自然语言更接近程序设计语言。它虽然不能直接执行但很容易被转换成高级语言

二、算法设计的原则

设计算法时，通常应考虑达到以下目标：

1．**正确性**

2. **可读性**

3．**健壮性**

4．**高效率与低存储量需求**

1．正确性

首先，算法应当**满足**以特定的“**规格说明**”方式给出的**需求**。

其次，对算法是否“**正确**”**的**理解可以有以下**四个层次**：

a．程序中不含语法错误；

b．程序对于几组输入数据能够得出满足要求的结果；

c．程序对于精心选择的、典型、苛刻且带有刁难性的几组输入数据能够得出满足要求的结果；

d．程序对于一切合法的输入数据都能得出满足要求的结果；

通常以第 **c** 层意义的正确性作为衡量一个算法是否合格的标准。

2. 可读性

算法主要是为了人的**阅读与交流**，其次才是为计算机执行，因此算法应该**易于人的理解**；另一方面，晦涩难读的程序易于隐藏较多错误而难以调试。

3．健壮性

当输入的数据**非法**时，算法应当恰当地作出反映或**进行相应处理**，而不是产生莫名奇妙的输出结果。并且，**处理出错的方法**不应是中断程序的执行，而应是**返回**一个**表示错误或错误性质的值**，以便在更高的抽象层次上进行处理。

4．高效率与低存储量需求

通常，效率指的是算法执行时间；存储量指的是算法执行过程中所需的最大存储空间，两者都与问题的规模有关。

有效的算法

- 一个算法如果能够在所要求的资源限制内(空间和时间)将问题解决好，称这个算法是有效的。一个算法的代价是算法消耗的资源量。
- 权衡的概念： 每个数据结构都有相关的代价和效率的衡量。
- 代价和效率：一个数据结构都将代价和效益联系在一起。一个数据结构需要一定的空间来存储它的每个数据项，一定的时间来执行单个基本操作和一定的程序设计工作。

三、算法效率的衡量方法和准则

通常有**两种**衡量算法效率的方法:

事后统计法

缺点：1．必须执行程序

2．其它因素掩盖算法本质

事前分析估算法

- 对同一个问题运行四种不同算法，所用时间依赖于：算法，问题规模，语言，编译器和计算机

second	Algorithm1	Algorithm2	Algorithm3	Algorithm4
N=100	0.000 159	0.000 006	0.000 005	0.000 002
N=1000	0.095857	0.000 371	0.000 060	0.000 022
N=10000	86.67	0.033 322	0.000 619	0.000 222
N=100000	NA	3.33	0.006 700	0.002 205
N=1000000	NA	NA	0.074 870	0.022 711

算法执行**时间**相关的**因素**

1. **算法**选用**的策略**
2. **问题的规模**
3. 编写程序的**语言**
4. **编译**程序产生的机器代码**的质量**
5. **计算机**执行指令**的速度**

一个特定**算法的“运行工作量”**的大小，只依赖于问题的规模（通常用整数量n表示），或者说，**它是问题规模的函数。**

例 问题规模通常可以从问题描述中找到

（1）在一个具有n个学生信息的文件中查找某个名叫张明的学生，该问题规模为n；

（2）对一个具有n个整数组成的数组进行排序，其问题规模为n。

second	1	2	3	4
N=100	0.000 159	0.000 006	0.000 005	0.000 002
N=1000	0.095857	0.000 371	0.000 060	0.000 022
N=10000	86.67	0.033 322	0.000 619	0.000 222
N=100000	NA	3.33	0.006 700	0.002 205
N=1000000	NA	NA	0.074 870	0.022 711

渐进分析法

- **渐进分析：忽略用来表示问题规模和时间关系的函数中不能显著改变函数数量级的部分，其结果是原函数的近似值，该近似值在问题规模很大时会足够接近原值。**

- **渐进分析是对资源开销的一种不精确的估计，它提供的是对算法所需资源开销进行评估的简单化模型，以便把注意力集中在最重要的部分。 例：$f(n)=n^2+100n+\log_{10}n+1000$**

函数$f(n)=n^2+100n+\log_{10}n+1000$中各项的增长速度

n	f(n)	n^2		100n		$\log_{10}n$		1000	
	值	值	%	值	%	值	%	值	%
1	1101	1	0.1	100	9.1	0	0.0	1000	90.83
10	2101	100	4.76	1000	47.6	1	0.05	1000	47.6
100	21002	10000	47.6	10000	47.6	2	0.001	1000	4.76
1000	1101003	1000000	90.8	100000	9.1	3	0.0003	1000	0.09
10000	101001004	100000000	99.0	1000000	0.99	4	0.0	1000	0.001
100000	10010001005	10000000000	99.9	10000000	0.099	5	0.0	1000	0.00

- 【关注】问题规模（实例特征）*n*足够大时程序的性能，即复杂度问题一般考虑的只是对于问题规模*n*的增长率。
- 在难以获得精确的操作计数时，只需求出它关于*n*的增长率即可。

(渐近)时间复杂度

O表示法:

$T(N) = O(f(N))$

如果存在一个正常数c 和 n_0， 当n>= n_0 时，总有T(N) <= cf(N)。

Ω 表示法

$T(N) = \Omega(g(N))$

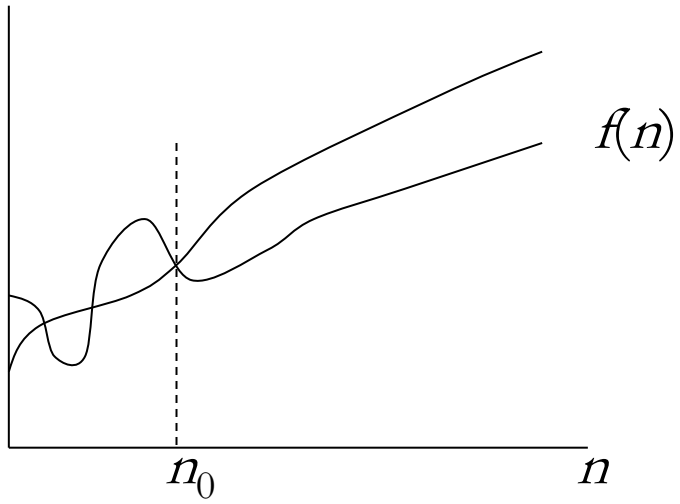
如果存在一个正常数c 和 n_0， 当n>= n_0 时，总有T(N) >= cg(N)。

定义---大O 表示法

$f(N) = O(g(N))$

如果存在一个正常数 c 和 n_0，当 $N >= n_0$ 时，一直存在

$f(N) <= cg(N)$



Big Oh notation $f(n) \in O(g(n))$

(渐近)时间复杂度

Θ定义:

$$T(N) = \Theta(b(N))$$

当且仅当$T(N) = O(b(N))$ 并且 $T(N) = \Omega(b(N))$

o定义

$$T(N) = o(p(N))$$

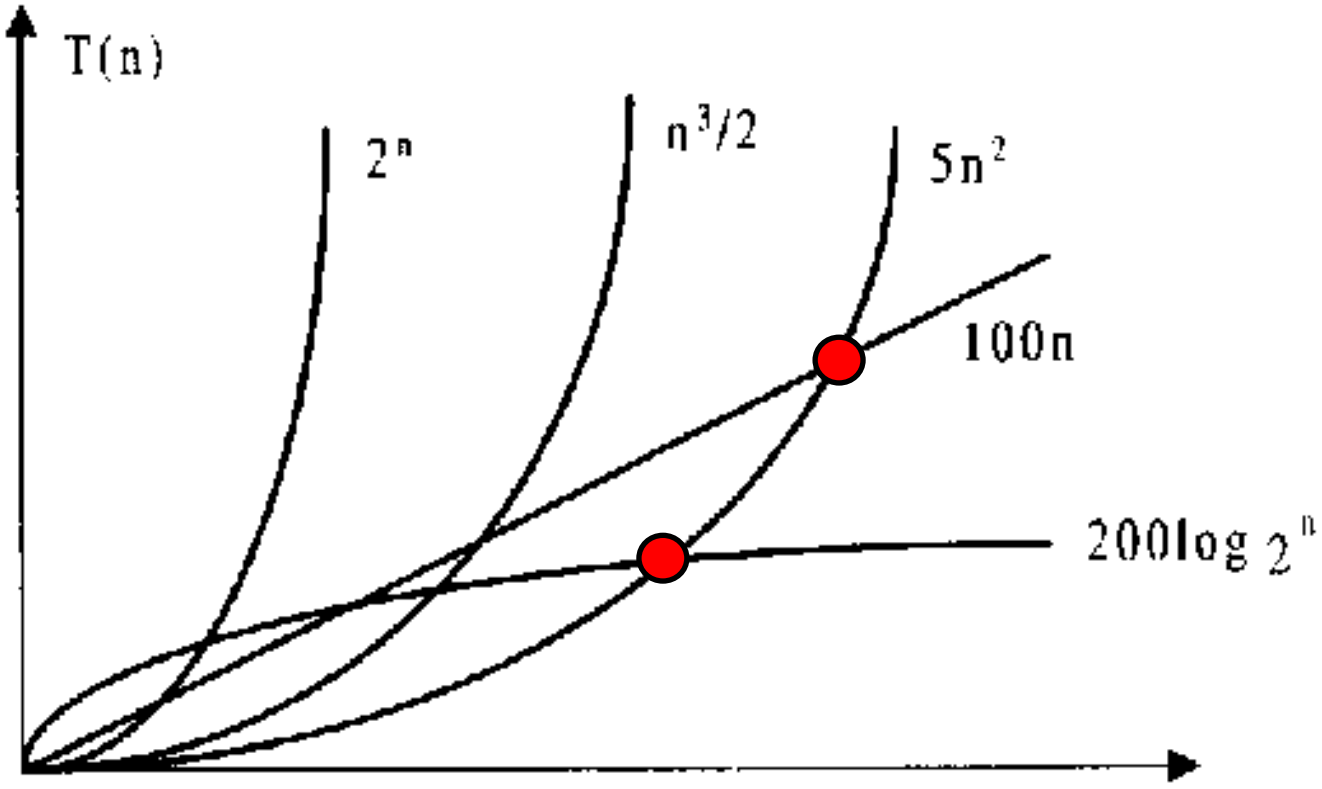
如果$T(N) = O(p(N))$，但是 $T(N) \neq \Theta(p(N))$

(渐近)时间复杂度

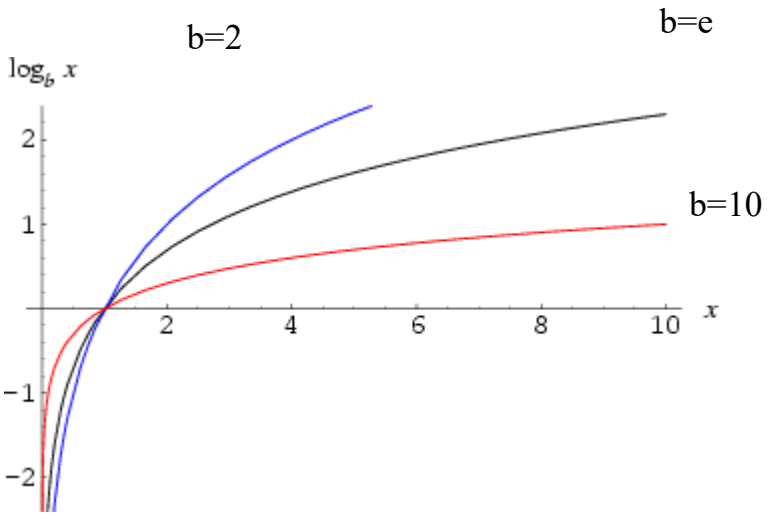
物理含义:

- 在某些点f(N) < g(N) 是没意义的
- 相对增长速度

(渐近)时间复杂度



(渐近)时间复杂度



(渐近)时间复杂度

Function	Name
c	Constant
logN	Logarithmic
$\log^2 N$	Log-squared
N	Linear
NlogN	
N^2	Quadratic
N^3	Cubic
2^N	Exponential

Typical Growth Rates

(渐近)时间复杂度

- 规则 1

如果 $T_1(N) = O(f(N))$ 且 $T_2(N) = O(g(N))$，那么

(a) $T_1(N) + T_2(N) = \max(O(f(N)), O(g(N)))$

(b) $T_1(N) \times T_2(N) = O(f(N) \times g(N))$

Examples:

如果 $T_1(N) = O(N^2)$ 且 $T_2(N) = O(N)$ 那么

(a) $T_1(N) + T_2(N) = O(N^2)$

(b) $T_1(N) * T_2(N) = O(N^3)$

(渐近)时间复杂度

- 规则 2

如果 $T(N)$ 是一个k阶多项式，那么

$$T(N) = \Theta(N^k)$$

规则 3:

对于任何常数k，$\log^k N = O(N)$。

也就是说对数函数增长非常缓慢。

(渐近)时间复杂度

在大O表示法中，应该忽略:

低阶项

常数项

例如:

correct form:$T(N) = O(N^2)$

bad style: $T(N) = O(2N^2)$

correct form:$T(N) = O(N^2)$

bad style: $T(N) = O(N^2+N)$

(渐近)时间复杂度

- Using L'Hopital's rule to evaluate:

$$\lim_{N\to\infty} \frac{f(N)}{g(N)}$$

(1) $\lim = 0; \quad \Rightarrow f(N) = o(g(N)).$

(2) $\lim = c \neq 0; \quad \Rightarrow f(N) = \Theta(g(N)).$

(3) $\lim = \infty; \quad \Rightarrow g(N) = o(f(N)).$

(4) limoscillates; $\Rightarrow$ no relation.

如何估算

算法的时间复杂度?

算法 = 控制结构 + 原操作

(固有数据类型的操作)

算法的执行时间 =

∑原操作(i)的执行次数×原操作(i)的执行时间

算法的执行时间

与

原操作执行次数之和 **成正比**

从算法中选取一种对于所研究的问题来说是 **基本操作** 的原操作，以该基本操作 **在算法中重复执行的次数** 作为算法运行时间的衡量准则。

例1：两个矩阵相乘

```
void mult(int a[], int b[], int& c[] ) {
// 以二维数组存储矩阵元素，c 为 a 和 b 的乘积
   for (i=1; i<=n; ++i)
      for (j=1; j<=n; ++j) {
         c[i,j] = 0; // 每个结果元素单独累加
         for (k=1; k<=n; ++k)
            c[i,j] += a[i,k]*b[k,j];
      } //for
} //mult
```

基本操作：**乘法**操作

时间复杂度：$O(n^3)$

例2：选择排序

```
void select_sort(int& a[], int n) {
  // 将a中整数序列重新排列成自小至大有序的整数序列。
    for ( i = 0;  i< n-1;  ++i ) {
        j = i;   // 选择第i个最小元素
        for ( k = i+1;  k < n;  ++k )
            if (a[k] < a[j] )  j = k; // j 保存当前最小值的位置
        if ( j != i )  a[j] ←→ a[i]
    }
} // select_sort
```

基本操作：

比较(数据元素)操作

时间复杂度：$O(n^2)$

例3: 起泡排序

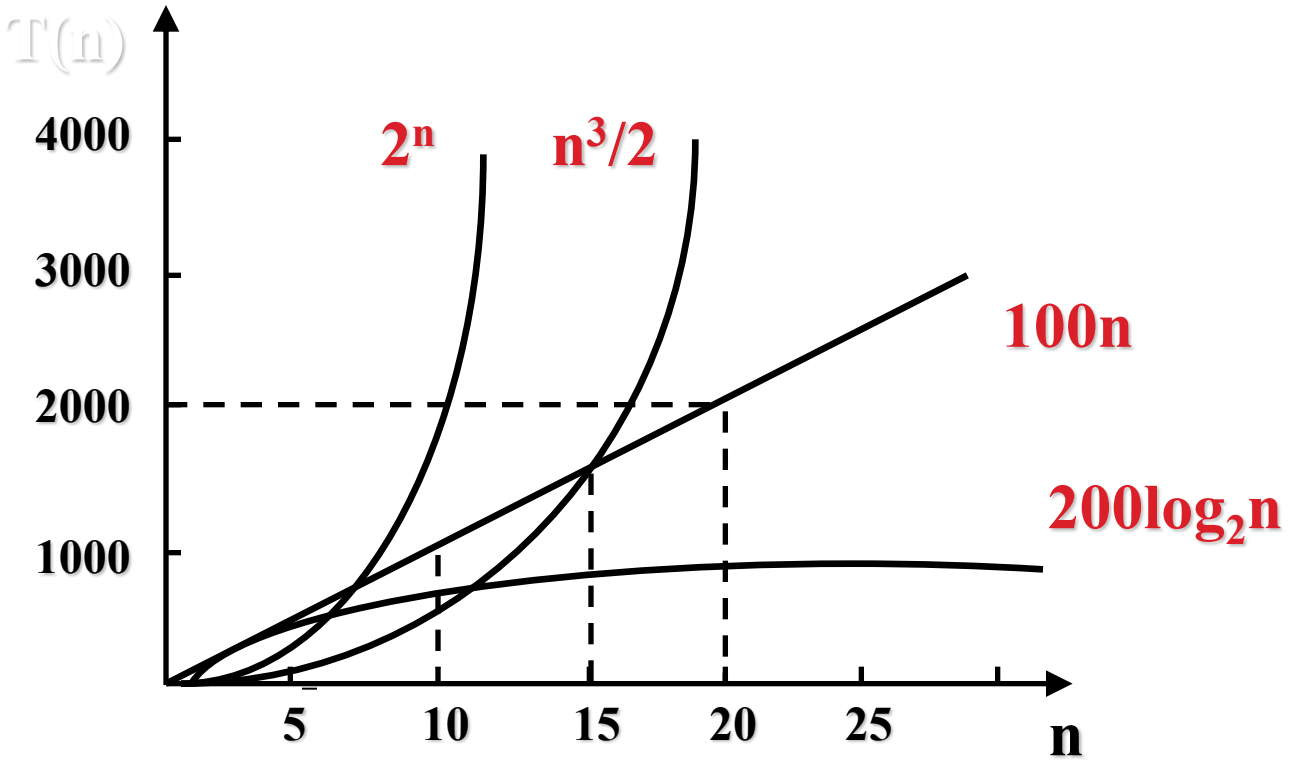
```
void bubble_sort(int& a[], int n) {
  // 将 a 中整数序列重新排列成自小至大有序的整数序列。
for (i=n-1, change=TRUE;  i>1 && change;  --i)
  {  change = FALSE;  // change 为元素进行交换标志
       for (j=0;  j<i;  ++j) // 只比较当前未排序区间
          if (a[j] > a[j+1])
             { a[j] ←→ a[j+1];   change = TRUE ;}
  } // 一趟起泡
} // bubble_sort
```

基本操作：**赋值操作**

时间复杂度：$O(n^2)$

-常用的渐进函数

函 数	名 称
O(1)	常量级
O(n)	线性级
$O(n^2)$	平方级
$O(n^3)$	立方级
O(logn)	对数级
O(nlogn)	n个logn
$O(2^n)$	指数级
O(n!)	阶乘级



常见函数的增长率示意图

- 一般地，常用的时间复杂度有如下关系：

$$O(1) \le O(\log n) \le O(n) \le O(n\log n) \le O(n^2) \le O(n^3)$$

$$\le \ldots \le O(n^k) \le O(2^n) \le O(3^n) \le O(n!)$$

-小结：程序（算法）渐进时间复杂度的计算

- 算法的渐进时间复杂度考虑的只是对于问题规模的增长率，在难以精确计算基本操作语句执行次数的情况下，只需求出它关于n的增长率或数量级即可。
- 算法的时间复杂度通常以算法中关键操作重复执行的次数作为度量标准；
- 被视作算法基本操作的一般是最深层循环内的语句。——算法的时间复杂度是由嵌套最深层语句的频度（该语句重复执行的次数）决定的。
- 这种衡量效率的办法所得出的不是时间量，而是一种增长趋势的量度。它与软硬件环境无关，只暴露算法本身执行效率的优劣。

(4) 对于某些算法复杂度，不仅与问题的规模有关，而且还与它所处理的数据集的状态有关。因此即使问题规模相同，如果输入数据不同，其时间开销也不同

- 最佳：没兴趣 。发生概率太小，条件考虑太乐观
- 最差：知道算法至少能多快。对实时系统尤其重要。
- 平均情况： 程序要对许多情况运行多次，通常希望知道平均情况的时间代价。平均情况分析并不总是可行的，它要求我们清楚数据的如何分布。

总之，在实时系统，**关注最差情况**。在其他情况，通常考虑平均情况，只要知道计算平均情况所需的输入数据的分部即可。否则，只能求助最差的情况分析。

【课堂练习】渐进时间复杂度分析

- 练习1　x++;

O(1)

- 练习2　for(i=0;i<n;i++)
  x++;

O(n)

- 练习3　for(i=0;i<n;i++)
  for(j=0;j<n;i++)
  x++;

$O(n^2)$

- **练习4**

```
for(i=1;i<n;i++)
{
    y=y+1;
    for(j=0;j<=2*n;j++)
        x++;
 }
```

解：语句x++的执行次数是：$(n-1)(2n+1)=2n^2-n-1$

该程序段的渐进时间复杂度为：$T(n)=O(n^2)$

◆ 练习 5

```
i=1;                    /*语句1*/
while(i<=n)
    i=i*2;              /*语句2*/
```

解： 设语句2的执行次数是：f(n)，

则有$2^{f(n)} \leq n$，

即$f(n) \leq \log_2 n \leq c*\log_2 n$，c为正常数

所以，该程序段的渐进时间复杂度为：$O(\log_2 n)$

四、算法的存储空间需求

算法的**空间复杂度定义为:**

$$S(n) = O(g(n))$$

表示随着问题规模 n 的增大，算法运行所需存储量的增长率与 g(n) 的增长率相同。

算法的存储量包括:

指令常数变量所占用的存储空间;
输入数据所占用的存储空间;
辅助(存储)空间。

一般地，算法的空间复杂度指的是辅助空间。

一维数组a[n]： 空间复杂度 O(n)
二维数组a[n][m]： 空间复杂度 O(n*m)

若**输入数据**所占空间只取决于问题本身，**和算法无关**，则只需要分析除输入和程序之外的**辅助变量**所占**额外空间**。

若所需额外空间相对于输入数据量来说是常数，则称此算法为**原地工作**。

若所需存储量依赖于特定的输入，则通常按最坏情况考虑。

